Squad 11

Arithmetic Parser for Larger Compiler
Software Development Plan
Version <1.0>

Bobby Clark

Jaiden Green

Nathan Le

Emily Messenger

Jeremiah Penney

Revision History

Date	Version	Description	Author
02/11/26	1.0	Filled out roles and responsibilities table (3.3) and project plan (4.2)	Honey Patel
2/14/26	1.0	Added details to project deliverable section (2.3)	Honey Patel
2/15/26	1.0	Added to Introduction (1), purpose (1.1), scope (1.2)	Honey Patel
2/16/26	1.0	Organizational Structure (3.1) and External interfaces (3.2)	Honey Patel
2/17/26	1.0	Iteration objectives (4.2.2)	Honey Patel
2/18/26	1.0	Overview (1.5), Releases (4.2.3) and completed Project Monitoring and Control (4.3). **highlighted potentially remaining sections in red	Honey Patel
2/19/26	1.0	Working as group in group meeting	
2/19/26	1.0	Added to Quality Control (4.5) and Configuration Management (4.8)	Emily Messenger
2/19/26	1.0	Added to Configuration Management (4.8)	Emily Messenger / Jeremiah
2/19/26	1.0	Setup & filled out 3.1 & 3.3	Jaiden Green
2/19/26	1.0	Evolution of the software development plan (2.4) and annexes (5)	Honey Patel
2/19/26	1.0	(1.4)(2.2)	Bobby

Table of Contents *[keep this; say N/A when inapplicable]*

1. Introduction 4

1.1 Purpose 4

1.2 Scope 4

1.3 Definitions, Acronyms, and Abbreviations 4

1.4 References 4

1.5 Overview 5

2. Project Overview 5

2.1 Project Purpose, Scope, and Objectives 5

2.2 Assumptions and Constraints 5

2.3 Project Deliverables 6

2.4 Evolution of the Software Development Plan 6

3. Project Organization 7

3.1 Organizational Structure 7

3.2 External Interfaces 7

3.3 Roles and Responsibilities 7

4. Management Process 8

4.1 Project Estimates 8

4.2 Project Plan 8

4.3 Project Monitoring and Control 10

4.4 Requirements Management 11

4.5 Quality Control 11

4.6 Reporting and Measurement 11

4.7 Risk Management 11

4.8 Configuration Management 12

5. Annexes 12

Software Development Plan

1. Introduction

This document outlines iteration and artifact/deliverable schedules regarding the development of an arithmetic expression evaluator in C++. The plan outlines the requirements specifications, software architecture, implementation, and test cases. Team roles/responsibilities are also assigned and meeting schedules are outlined.

1.1 Purpose

Our Software Development Plan defines the development process of the arithmetic expression evaluator. It highlights the functional/nonfunctional requirements and defines the constraints under which the software will be developed and maintained. Additionally, we clearly map out the schedule of each phase and assign roles to each team member.

The purpose of the *Software Development Plan* is to gather all the information necessary to control the project. It describes the approach to the development of the software and is the top-level plan generated and used by managers to direct the development effort.

The following people use the *Software Development Plan*:

- The **project manager** uses it to plan the project schedule and resource needs, and to track progress against the schedule.
- **Project team members** use it to understand what they need to do, when they need to do it, and what other activities they are dependent upon.

1.2 Scope

This *Software Development Plan* describes the overall plan to be used by the arithmetic expression evaluator project, including deployment of the product. The plan describes the development process, the schedule, artifact release schedule, and team structure. The details of the individual iterations will be described in the Iteration Plans.
The plans as outlined in this document are based upon the product requirements as defined in the *Vision Document*.

1.3 Definitions, Acronyms, and Abbreviations

See the Project Glossary in glossary section of References (1.4)

1.4 References

Iteration Plans – Implement first assuming integer inputs and operator support to ensure proper expression parsing. Float point inputs and parenthesis handling will be adapted later in the process. Test cases will be used to debug the program, allowing for accuracy in the final version.

Vision – A user friendly program that can correctly interpret and evaluate mathematical expressions with +, -, *, /, %, and **. It will handle exceptions by prompting the user with an error, rather than crashing.

Glossary –

Project Deliverable – Main parts of significant sections of the UPEDU that are worked on and submitted to the repository that can be revised with proper documentation of these revisions

Arithmetic Expression – An expression that's a combination of operands (constants, literals, variables, etc…) and operators (+,-,/,etc…) that can be calculated in a specific way (PEMDAS) to get the

Parsing – An algorithm that takes in a sequence of tokens and decides whether that sequence is valid/ runnable.

Tokenizer – The first stage of a compiler pipeline that translates source code into something a computer can execute.

Token – The smallest unit of information that has meaning in a program. Keywords, Identifiers, Variables, Operators, Punctuation, Separators

UPEDU – Unified Process Education, a software engineering model designed from the Unified Process to be a shorter version to be used for education purposes

Nonfunctional Requirements – Requirements beyond what the system should do. How fast the program should run, how much traffic it should handle, how it should look, what timeframe it should be done by, budget, etc…

Functional Requirements – Requirements on how the system should work, things that it should be able to handle, do, and calculate. ("WHAT the system does")

1.5 Overview

This *Software Development Plan* contains the following information:

Project Overview — Defines the purpose, functional requirements, non-functional requirements, and constraints of the system. The project schedule is outlined in detail, with key dates for iterations and deliverables.

Project Organization — Lists team members' names, contact information, and availability. Each member's role is listed, alongside their specialties/expertise.

Management Process — Time effort needed for each iteration is estimated, alongside expected competition dates. Project updates are monitored in the meeting minutes, allowing for consistency throughout the document.

Applicable Plans and Guidelines — Software architecture, and developmental environment are outlined. Resources needed for each phase are also listed, alongside how they are applicable to the development of each iteration.

2. Project Overview

2.1 Project Purpose, Scope, and Objectives

To Build a versatile arithmetic expression evaluator using C++. The program will take an arithmetic expression as input, parse it, and calculate the result according to PEMDAS and associativity rules. Expressions could include the operators +, -, *, /, %, and **, along with numeric constants. Additionally, the program must correctly interpret expressions with parentheses to establish precedence and grouping.

2.2 Assumptions and Constraints

Initially assume inputs are integers. Coding in C++. The module receives characters sequentially. Source character set as ASCII/UTF-8. Arithmetic Input. Assume PEMDAS for operator precedence. Interface should be user friendly with a legible command line interface that allows users to enter expressions and display calculated results. Object oriented programming used. Using UPEDU.

2.3 Project Deliverables

Deliverable 1: Project Plan, due February 22

Map out the scope of the project (integer definitions), team roles, and work progress timelines. Create GitHub repository.

Deliverable 2: Software Requirements Specification, due March 15

Outline the functional requirements needed for the parser like how it should support expression parsing, operators, unary, parenthesis handling, and recognize numeric constants. Nonfunctional requirements include robustness/error handling, code quality, and accuracy. Constraints include C++ coding environment and use of Linux environment.

Deliverable 3: Software Architecture, due April 5

Map out use cases (user inputs expression--->input tokenized--->tokens are parsed--->expression is evaluated--->output displayed)

Deliverable 4: Implementation, due May 7

Establish development environment, finalize GitHub repositories, and begin building the parser. Ensure expressions accurately evaluated, supports unary operators and parentheses, supports error handling, build makefile, and include documentation and comments.

Deliverable 5: Test Cases, due May 7

Specify hardware characteristics and configuration, specify system software (operating systems, compilers, simulators, test tools) needed for test cases, define test case constraints, and list test case intercase dependencies.

Deliverable 6: User Manual, May 7

Manual that defines how to install, configure, and use the calculator software. Manual should be easy to navigate/understand and provide introduction, usage guide, describe advanced features, and provide troubleshooting.

2.4 Evolution of the Software Development Plan

The *Software Development Plan* will be revised prior to the start of each Iteration phase. Any subsequent edits after 2/22 will be recorded with revisions under Revision History and be noted as unscheduled revision.

Version	Date	Description

3. Project Organization

3.1 Organizational Structure

The organizational structure focuses on splitting up even amounts of work to complete on our own time. At meetings, we can review what work has been done and what still needs to be completed. We will emphasis coordination and consistency by updating meeting minutes and tracking document changes. The assigned roles below ensure that we each contribute to the development of the project in an efficient manner.

Jaiden - GitHub Management

Bobby - Management

Emily – Quality Control

Nathan – Debugger

Honey – doc writer / Organization

Jeremiah- note taker / doc over viewer

3.2 External Interfaces

Professor: Hossein Saiedian

Responsibilities: Assigns the project, provides clarifications, defines requirements and ensures the project meets them.

Teaching Assistant: Akintoye Oloko

Responsibilities: Provides guidance during developmental process, grades manuals/documents, reviews documentation, verifies software against test cases, grades overall project.

Students: Bobby, Nathan, Honey, Jaiden, Emily, Jeremiah

Responsibilities: Define the requirements and software architecture to develop the software through implementation and testing. Submit deliverables and the final project in a timely manner.

3.3 Roles and Responsibilities *[the more details here, the easier your job; include contact info, availability info, expertise, …]*

Person	Unified Process for EDUcation Role
Bobby (972-469-3991), management, has class Friday 11:00-12:00, Thursday 9:00-3:15 and every other day 11:00-3:15	Tester Expertise: Python, Math
Nathan (913-777-9711), debugger, available Wednesday after 2 pm	Analyst, Implementor Expertise: Python, debugging
Honey (913-337-6423), organization/timekeeping, available Mondays/Wednesday after 5, and Thursday 3:30	Reviewer, Change control manager Expertise: Python, organization
Jaiden (913-530-3977), GitHub	Configuration Manager

management, available Monday/Wednesday 2-4:30	Experience: Python, cpp, c, cybersecurity, full-stack development
Emily (913-481-0094), quality control, available after Wednesday 3:15	Implementor Expertise: Java, Python, OOP, quality control
Jeremiah (913-601-1863), notetaker/overviews doc available Mondays/Wednesday after 5, and Thursday 3:30	Designer, Project Manager Expertise: Hardware interface, C++, C

Weekly meetings on Wednesday in Anschutz Library Study Room from 3:30-5:30. Communication is done through iMessage group chat.

Anyone on the project can perform Any Role activities.

4. Management Process

4.1 Project Estimates

Phase	Duration	Main Activities
Requirements & Planning	1 week	Study project description, define requirements, prepare SRS
Architecture & Design	1 week	Design parsing approach, class structure, and architecture
Implementation	2 weeks	Implement tokenizer, parser, evaluation logic, error handling
Testing & Documentation	2 weeks	Develop test cases, verify results, prepare user manual and final documentation

Estimated effort distribution:

- Requirements and planning: 10 hours
- Architecture and design: 10 hours
- Implementation: 25 hours
- Testing and documentation: 15 hours

Re-estimation will occur at each weekly meeting. Progress and setbacks will be taken into consideration to re-estimate the timeline for the project.

4.2 Project Plan

**Due dates differ from the dates we will aim for

Deliverable 1: Project Plan – Due Feb 22 – 11:59 pm

-resources: Microsoft Word, UPEDU project plan template, GitHub, communication

Deliverable 2: Software Requirements Specification – Due March 15, 11:59 pm

-resources: Microsoft Word, UPEDU SRS template, GitHub, UML diagram/tools

Deliverable 3: Software Architecture – Due April 5, 11:59 pm

-resources GitHub, software architecture, UPEDU software architecture template, UML tools

Deliverable 4: Implementation – Due May 7, 11:59

-resources: IDE, GitHub, UPEDU test case specification template, debugging tools

Deliverable 5: Test Cases – Due May 7, 11:59

-resources: testers, operating system, IDE, GitHub, Microsoft Word, UPEDU test case specification template

Deliverable 6: User Manual – Due May 7, 11:59

-resources: Microsoft word, GitHub, IDE, pictures of software,

Iteration 1: finalize project scope, assign team roles, and project planning by Feb 22

Iteration 2: Establish functional requirements, nonfunctional requirements, and constraints through establishing a software requirement specification by March 15

Iteration 3: Establish a system architecture and map out use case modeling diagrams by April 5

Iteration 4: Establish the development environment and begin developing the tokenizer and parser. Focus on accurate expression evaluation and account for edge cases. April 20

Iteration 5: Build the makefile and begin defining test case constraints and intercase dependencies. Verify the software against the test cases to check for functionality. Conduct debugging April 27

Iteration 6: Finalize user manual and write out documentation, comments, and conduct final review. May 5

4.2.1 Phase Plan

[Include the following:

- *a Gantt chart showing the allocation of time to the project phases (Not necessarily detailed to the activity level; this type of Gantt Chart is providing along with the Iteration Plans themselves; Provide an Overview of the project Timeline with the major miles stones]*
- *identify **major milestones** with their achievement criteria*

Define any important release points and demos.]

*[If available, refer to the related **Iteration Plan Documents** for more details]*

4.2.2 Iteration Objectives

Iteration 1 objective: Finalize the project's scope, goal, and requirements. Assign the team roles and establish a meeting schedule. Complete the UPEDU project plan template.

Iteration 2 objective: Create a software requirements specification template that clearly defines the requirements and constraints of the calculator. Outline the purpose of the requirements and how they fit with each other and within the system

Iteration 3 objective: Establish the software architecture and identify its components, like input module, tokenizer module, parser module, evaluation module, and user interface module.

Iteration 4 objective: Establish the development environment and begin to build the calculator. Configure GitHub pages.

Iteration 5 objective: Document test cases that validate the functionality of the system by testing functional and non-functional requirements.

Iteration 6 objective: Create a user manual that explains how to install, configure, and use the calculator software. Include troubleshooting steps and describe advanced features of the software.

4.2.3 Releases

Demo release: April 20

Initial system is implemented with tokenizer and parser evaluating various integer expressions

Beta Release: April 27

Complete version is released after it has been tested against differing cases, ensuring that the evaluator can accurately parse and handle parentheses, unary operators, and numeric constants.

Final Release: May 5

The final and tested version of the evaluator has been released. The complete makefile, documentation, and user manual is submitted through GitHub.

4.2.4 Project Schedule

Phase	Description	Start Date	End Date	Milestone
Requirements	Gather and finalize system requirements	Feb 12	Feb 22	Requirements Approved
Design	Create system architecture and design documentation	Feb 22	Mar 15	Design Review Complete
Implementation	Develop and integrate system components	Mar 15	Apr 20	Feature Complete
Testing	Perform unit testing and system testing	Apr 20	Apr 27	Testing Complete
Final Review & Submission	Final demo and documentation submission	Apr 27	May 5	Project Delivered

4.2.5 Project Resourcing

[Identify the numbers and type of staff required here, including any special skills or experience, scheduled by project phase or iteration.

List any special training project team members will require, with target dates for when this training should be completed.]

4.3 Project Monitoring and Control

Requirements Management: Requirements will be tracked using the project management document, listing functional and non-functional requirements. Any proposed changes (such as adding floating-point support or new operators) must be submitted to the team, discussed during meetings, and approved by consensus.

Quality Control: Quality will be assured through rigorous testing of code blocks, peer review of code during/ outside of meetings. The team member in charge of quality control will be the confirmation of quality in code, where it should run smoothly, correctly, and is able to handle possible exceptions.

Risk Measurement: Risks will be tracked through the meeting log. Members will bring up any risk or issues found during meetings; these will stay in the “Last Meeting” section until they are handled to ensure that nothing is left to the end to be fixed. During meetings, the logistics of how to solve each issue will be discussed and recorded.

Configuration Management: The project will be managed through GitHub. Through this, we will be able to manage different versions of the system and track new features added. Project artifacts will be labeled in a consistent manner where we will include the name of the file, and its version number. Commits will include

detailed comments on what the code is doing. Through GitHub's backup feature, we will be able to recover any lost data/progress by restoring previous versions.

4.4 Requirements Management

The requirements for this system are captured in the Project Monitoring document.

4.5 Quality Control

Defects will be identified via a comprehensive review process performed after any new functionality is developed. Appropriate tests and documentation (e.g., checklists) will aid in this process and quantify the efficiency and accuracy of the software.

This will maintain the quality of the software and streamline the debugging process. Any resulting changes will be monitored in the Project Monitoring Document to track progress and ensure that defects aren't neglected.

4.6 Reporting and Measurement

Updated schedule estimates, and metrics summary reports, will be generated at the end of each iteration.

The Minimal Set of Metrics, as described in the RUP Guidelines: Metrics will be gathered on a weekly basis. These include:

Earned value for completed tasks. This is used to re-estimate the schedule and budget for the remainder of the project, and/or to identify need for scope changes.

Total defects open and closed – shown as a trend graph. This is used to help estimate the effort remaining to correct defects.

Acceptance test cases passing – shown as a trend graph. This is used to demonstrate progress to stakeholders.

Refer to the Project Measurements Document (AAA-BBB-X.Y.doc) for detailed information.

4.7 Risk Management

Identified Project Risks

1. Incorrect Operator Precedence Implementation
   - The evaluator may incorrectly implement PEMDAS rules or associativity
2. Parentheses Parsing Error
   - Mismatched or nesting parentheses may not be handled correctly.
3. Unary Operator Ambiguity
   - Difficulty distinguishing unary from binary subtraction
4. Division by Zero Error Handling
   - Program fails to detect user dividing by zero
5. Incomplete Test Coverage
   - Insufficient test cases may fail to capture edge conditions
6. Schedule Risks

- Parsing logic may require more development time than planned

7. Scope Expansion
   - Future change request may require floating-point support
8. Team Coordination
   - Conflicts in code integration or inconsistent documentation updates

4.8 Configuration Management

A GitHub repository will be used to maintain and organize relevant files (e.g., source code, test scripts). README documents associated with each of these files will detail the purpose and procedures for software. Moreover, projects' artifacts as well as documentation will be kept in the repository to make sure that we properly keep track of different changes.

5. Annexes

UPEDU templates will be used to complete the deliverables. C++ programming and makefile guidelines will be used. All files will be in GitHub and maintained through version control.